Sprint 1 Requirements Artifacts

Sprint 1 scope

Sprint 1 delivers 14 requirements worth 50 story points, about 7.1 points per team member, covering the foundation every later feature depends on: deployment, accounts, households, invite codes and the app shell.

Team 12: Jaydee Brown, Aidan Bergeman, Didier A Carrillo, Will Godderz, Ethan Gao, Lea Sulthana Syed, Zackary Rhodes

Req ID	Requirement	Story points	Area
1	GitHub repository with branch and pull request rules	2	Setup
2	Project skeleton (React, ASP.NET Core, Python, PostgreSQL)	3	Setup
3	Cloud-hosted database and server on Render	5	Setup
88	CI/CD pipeline deploying on merge to main	5	Setup
4	Register with email and password	5	Accounts
5	Log in and log out securely	5	Accounts
6	Passwords stored hashed and salted	2	Accounts
7	Create a household	3	Households
8	Invite others with a code or link	5	Households
9	Join a household with an invite code	3	Households
10	Home dashboard	5	App shell
29	Forms reject invalid amounts, dates and missing fields	3	App shell
32	Navigation menu	2	App shell
56	Dark mode option	2	App shell
Total		**50**	

Each artifact below lists what "done" means, how a GTA can check it, and the evidence the team will attach at the Sprint 1 release.

Setup and infrastructure

Req 1: GitHub repository with branch and PR rules (2 pts)

Done when: all seven members can push branches to the shared repo, and main cannot be changed except through an approved pull request.

- main has a branch protection rule requiring a pull request and at least 1 approving review.
- Direct pushes to main are blocked.
- Every member is listed as a collaborator.
- The README lists the team, project name and how to run the app locally.

How a GTA verifies: open the repo, then Settings → Branches (or a screenshot of it) to see the rule; open the closed pull requests to see reviews before merges.

Evidence: repo link, screenshot of the branch protection rule, link to one reviewed and merged PR.

Req 2: Project skeleton (3 pts)

Done when: a fresh clone runs the React frontend, ASP.NET Core API and Python service locally with documented commands.

- The repo has `/frontend` (React), `/backend` (ASP.NET Core) and `/services` (Python) folders.
- Each part starts with one command listed in the README.
- The React app calls a C# health endpoint and shows the result on screen.
- The C# API connects to PostgreSQL and the Python service responds on its own health endpoint.

How a GTA verifies: clone the repo, follow the README, open the local frontend and see "API status: OK."

Evidence: folder structure screenshot, README link, screenshot of the running app showing the health check.

Req 3: Cloud-hosted database and server (5 pts)

Done when: the frontend, C# API, Python service and PostgreSQL database all run on Render, and data saved on one device appears on another.

- The frontend is deployed as a Render static site with a public URL.
- The C# API and Python service are deployed as Render web services.
- PostgreSQL is a Render database, with its connection string in environment variables, not in code.
- A record created from a phone shows up on a laptop after a refresh.

How a GTA verifies: open the public URL on two devices, create a test record on one, refresh the other.

Evidence: public app URL, screenshot of the Render dashboard with all four services live, screenshot of environment variables with values hidden.

Req 88: CI/CD pipeline (5 pts)

Done when: merging an approved pull request into main automatically builds, tests and deploys the app to Render.

- A GitHub Actions workflow runs on every pull request and builds the frontend and backend.
- A failed build blocks the pull request from merging.
- A merge to main triggers a Render deploy without manual steps.
- The deployed site shows the change within about 10 minutes.

How a GTA verifies: open the Actions tab and see green runs on recent PRs; compare the latest merge time with the Render deploy log.

Evidence: link to the workflow file, screenshot of a passing Actions run, screenshot of the matching Render auto-deploy.

Accounts

Req 4: Register with email and password (5 pts)

Done when: a new visitor can create an account from the sign-up page and is logged in afterward.

- Sign-up asks for name, email, password and password confirmation.
- A valid email and a password of 8+ characters creates the account and opens the dashboard.
- An email that is already registered shows "An account with this email already exists."
- Empty fields, a bad email format or mismatched passwords show an inline error, and no request is sent.

How a GTA verifies: on the live site, register `test1@example.com`; register the same email again and see the error; submit with an empty password and see the inline error.

Evidence: screenshots of the sign-up form, the success redirect and each error, plus a link to the PR.

Req 5: Log in and log out securely (5 pts)

Done when: a registered user can log in, stay logged in across page loads, and log out so protected pages are no longer reachable.

- Correct email and password opens the dashboard and issues a session token.
- A wrong password shows "Incorrect email or password" without saying which part was wrong.
- The token is sent with each API call, and the C# API rejects requests without a valid token (HTTP 401).
- Log out clears the token; visiting `/dashboard` afterward redirects to the login page.

How a GTA verifies: log in with the test account, refresh the page and stay logged in, log out, then type `/dashboard` into the address bar and land on login.

Evidence: screenshots of login, the wrong-password error and the post-logout redirect; screenshot of a 401 response from the browser's network tab.

Req 6: Passwords stored hashed and salted (2 pts)

Done when: no plain-text password exists anywhere in the database or logs.

- Passwords are hashed with a salted algorithm (for example ASP.NET Core Identity's PasswordHasher or BCrypt) before saving.
- The Users table stores only `passwordHash`, matching the class diagram.
- Two users with the same password have different stored hashes.
- Passwords never appear in API responses or server logs.

How a GTA verifies: view a screenshot of the Users table with two test accounts that share a password and see two different hashes.

Evidence: database screenshot, link to the hashing code, a unit test showing login still succeeds against the stored hash.

Households

Req 7: Create a household (3 pts)

Done when: a logged-in user can create a named household and becomes its admin.

- "Create household" asks for a name of 1–50 characters.
- The new household is saved with the creator as its first member and Admin.
- The creator is taken to that household's dashboard, showing the household name.
- An empty name shows an inline error and nothing is saved.

How a GTA verifies: log in, create "Test House," and see the dashboard titled Test House with the user listed as Admin.

Evidence: screenshots of the create form and the new dashboard; screenshot of the Household and membership rows in the database.

Req 8: Invite others with a code or link (5 pts)

Done when: a household member can generate an invite code and shareable link that expire and can be revoked.

- "Invite roommate" generates a unique code (for example 8 characters) and a link containing it.
- Each code stores an expiration time (default 7 days) and an active flag, matching the InviteCode class.
- A "Copy" button copies the code or link to the clipboard.
- The admin can revoke a code; a revoked code stops working right away.

How a GTA verifies: generate a code, copy the link, then revoke the code and confirm the revoked code is rejected when used (Req 9).

Evidence: screenshots of the invite screen with a code, the copied link and the revoked state.

Req 9: Join a household with an invite code (3 pts)

Done when: a second user can enter a valid code and see the shared household, following the sequence diagram.

- Entering a valid, active, unexpired code adds the user to the household and opens its dashboard.
- An invalid, expired or revoked code shows "This invite code is not valid. Ask your roommate for a new one."
- A user already in the household is told they are already a member, with no duplicate row.
- The household's member list shows the new roommate.

How a GTA verifies: with a second test account, enter the code from Req 8 and see Test House; try a made-up code and see the error.

Evidence: screenshots of the join screen, the success dashboard, the error message and the updated member list.

App shell

Req 10: Home dashboard (5 pts)

Done when: after login or joining, members land on a dashboard showing their household at a glance.

- The dashboard shows the household name, member list and cards for Balances, Chores, Groceries and Events.
- Cards for features coming in later sprints show a friendly empty state such as "No chores yet."

- Members can add, mark complete and remove dashboard items (title, assignee, completed), matching the DashboardItem class.
- The layout works at phone width (about 375 px) and desktop width.

How a GTA verifies: log in to Test House, add an item, mark it complete, remove it; resize the browser to phone width.

Evidence: desktop and phone-width screenshots; screenshots of an item added, completed and removed.

Req 29: Forms reject invalid input (3 pts)

Done when: every Sprint 1 form blocks bad input in the browser and the API rejects it too.

- Required fields cannot be submitted empty.
- Emails must match a valid format; dates must be real dates.
- Amount fields reject negative numbers and non-numbers, ready for expenses in Sprint 2.
- The C# API returns HTTP 400 with a message if invalid data bypasses the browser.

How a GTA verifies: on sign-up, household and dashboard forms, try an empty field, a bad email and a negative number; each shows an inline error.

Evidence: screenshots of each error; a screenshot of a 400 response from the API (for example via Postman).

Req 32: Navigation menu (2 pts)

Done when: a logged-in user can reach every main page from a menu on any screen.

- The menu links to Dashboard, Expenses, Chores, Groceries, Calendar and Settings.
- Pages not built yet show a "Coming in a later sprint" placeholder.
- The current page is highlighted.
- On phone width the menu collapses into a menu button.

How a GTA verifies: click each menu link on desktop, then open the collapsed menu at phone width.

Evidence: desktop and mobile screenshots of the menu.

Req 56: Dark mode (2 pts)

Done when: a user can switch between light and dark mode, and the choice is remembered.

- A toggle in the header or Settings switches every page between light and dark themes.
- The choice is saved to the user's dark mode field, matching the class diagram, and restored on next login.
- Text stays readable in both modes.

How a GTA verifies: turn on dark mode, log out, log back in and see dark mode still on.

Evidence: side-by-side light and dark screenshots of the dashboard.